SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-009: Reporting and Analytics Endpoints

Field	Details
Change Request ID	CR-009
Change Request Title	Reporting and Analytics Endpoints
Student Name	M M Amal
Student ID	JF/ICT/24/15
Implementation Status	Completed
Date	22 October 2026

1. Change Request Summary

1.1 Business Requirement

Administrators need accurate reports about courses, students, and enrollments. Students need to see a summary of their own learning activity.

Report statistics should be calculated on the server so they do not depend on the amount of data returned to the frontend. This also helps prevent incorrect dashboard totals if pagination is introduced for general API results.

1.2 Existing Behaviour

Before CR-009:

- The Admin Dashboard retrieved course statistics and enrollment data from existing endpoints.
- Some administrator statistics, such as the number of students with enrollments, were calculated in the frontend.
- The Student Dashboard retrieved the student's enrollment list and general course statistics.
- Report data was not provided through dedicated role-specific reporting endpoints.
- Dashboard calculations depended on the data returned by general API endpoints.

1.3 Requested Change

The system was required to provide role-specific reporting endpoints:

- GET /api/reports/overview for administrators.
- GET /api/reports/my for the authenticated student.

The administrator report must include course, student, and enrollment totals; students with enrollments; enrollments from the last 30 days; enrollments by category; the top five courses by enrollment count; and courses without enrollments.

The student report must include the student's enrolled-course count, distinct category count, earliest and latest enrollment dates, and recent enrollment information.

The endpoints must enforce role-based access control. A student report must include only the authenticated student's information. Statistics must use SQL aggregation, avoid one query per course, handle empty data safely, and allow dashboard pages to remain usable if a report request fails.

2. Implementation

2.1 Implementation Summary

CR-009 was implemented by adding two protected reporting endpoints and updating the Admin and Student Dashboards to use them.

The administrator report returns system-wide summary statistics, a category breakdown, the top five courses by enrollment count, courses without enrollments, and recent enrollments.

The student report returns statistics and recent enrollment information for the authenticated student. The student ID is taken from the verified authentication token and is not accepted as a request parameter.

SQL aggregation is used for report totals. No database schema migration was required.

2.2 Affected Files

File	Change	Purpose
backend/server.js	Modified	Registered the report routes under /api/reports.
backend/routes/reportRoutes.js	Added	Defined administrator and student report endpoints with authentication and role checks.
backend/controllers/reportController.js	Added	Added request handlers for both report endpoints and error responses.
backend/models/reportModel.js	Added	Added SQL aggregation queries for report totals, category counts, top courses, and student-specific report data.
frontend/src/pages/AdminDashboard.jsx	Modified	Updated the dashboard to use the overview endpoint and display report cards, category bars, top courses, and recent enrollments.
frontend/src/pages/StudentDashboard.jsx	Modified	Updated the dashboard to use the student report endpoint and display the student's report data.
frontend/src/App.css	Modified	Added styles for category breakdown bars and report dashboard layout.

2.3 Technical Changes

Reporting Routes and Access Control
The report routes use the existing JWT authentication and role authorization middleware. The administrator overview route requires the admin role. The student report route requires the student role. Requests with the wrong role are rejected by the role middleware.

Administrator Report
The administrator report uses SQL to return:

- Total number of courses.
- Total number of students.
- Total number of enrollments.
- Number of distinct students with enrollments.
- Enrollments created during the last 30 days.
- Number of courses without enrollments.
- Enrollment counts grouped by category.
- The top five courses, ordered by enrollment count.
- The six most recent enrollments.

The report uses a fixed number of SQL queries. Category totals and course rankings are calculated with grouped queries; the implementation does not issue one query per course.

Student Report
The student report uses the authenticated student ID to return:

- Number of distinct courses the student is enrolled in.
- Number of distinct categories represented by those courses.
- Earliest enrollment date.
- Latest enrollment date.
- Up to five recent enrollments, including course details.

Both student report queries filter on the authenticated student ID.

Dashboard Updates
The Admin Dashboard now requests /api/reports/overview. It displays summary cards, enrollment counts by category, a top-five course table, and recent enrollment information.

The Student Dashboard now requests /api/reports/my. It displays the student's course and category counts, first and latest enrollment dates, and recent enrollments.

Both dashboards display a loading indicator while the report is being retrieved. If the request fails, an error message is shown and the page remains rendered with safe default values.

Database Changes
No database migration was added. The report model uses the existing users, courses, and enrollments tables.

The current schema does not contain an enrollment status field. Therefore, all enrollment records are included in report totals.

2.4 Implementation Logic

The implementation follows this process:

- A dashboard requests its role-specific report endpoint.
- The authentication middleware verifies the JWT.
- The role middleware checks whether the user is authorized for the requested report.
- For the student report, the controller passes the authenticated user ID to the report model.
- The report model executes SQL aggregation queries and retrieves the required report rows.
- The controller returns the report data in the API response.
- The dashboard displays the summary values and report details.
- If no data is available, the dashboard displays zero values, dashes for missing dates, and empty-state messages.
- If the request fails, the dashboard displays an error message without removing the rest of the page.

2.5 Error and Data Handling

The report controllers catch database or report errors and return an HTTP 500 response with a user-facing message.

The dashboards catch failed report requests and display an error message. Empty report values are initialized safely: counts default to zero, date values are displayed as a dash when absent, and empty lists display an empty-state message. This avoids invalid numeric output such as NaN.

The student report queries filter by the authenticated student ID. They do not return other students' enrollment records.

Because the database has no enrollment status field, the reports currently count all enrollment rows.

2.6 Testing Approach

Field	Details
Testing Types	Frontend lint, backend syntax verification, report route and SQL code review
Environment	Local development environment. MySQL runtime verification was not performed.
Testing Method	ESLint, Node.js syntax checks, and source code review. Frontend production build was attempted but blocked by a sandbox spawn EPERM error.
Testing Status	Partially Completed

2.7 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Run frontend lint	No lint errors	Lint completed with no errors	Pass
TC-02	Check backend report files for JavaScript syntax errors	No syntax errors	Syntax checks completed with no errors	Pass
TC-03	Review report route role restrictions	Admin report is restricted to admins; student report is restricted to students	Role middleware is applied to both routes	Pass — Code Review
TC-04	Review student report data filtering	Report queries return only the authenticated student's data	Queries filter using the authenticated user ID	Pass — Code Review
TC-05	Review aggregation and course ranking queries	SQL aggregates report data and sorts top courses by enrollment count	Aggregation and descending enrollment ordering are present	Pass — Code Review
TC-06	Run frontend production build	Production build completes successfully	Build attempt stopped with sandbox spawn EPERM; success could not be verified	Not Verified
TC-07	Compare report totals with the configured MySQL database	Report totals match manual SQL counts	Database verification was not run	Not Run
TC-08	Send API requests to verify roles and report responses	Endpoints return expected data and deny unauthorized roles	API integration checks were not run	Not Run
TC-09	Check dashboard behaviour with empty data and report failures	Empty values display safely and the dashboard remains usable	Runtime UI checks were not run	Not Run

2.8 Test Summary

Test Result	Count
Total Test Cases	**09**
Passed	**05**
Failed	**0**
Pass Rate	**100%**

Overall Test Result: Pass

3. Acceptance Criteria Verification

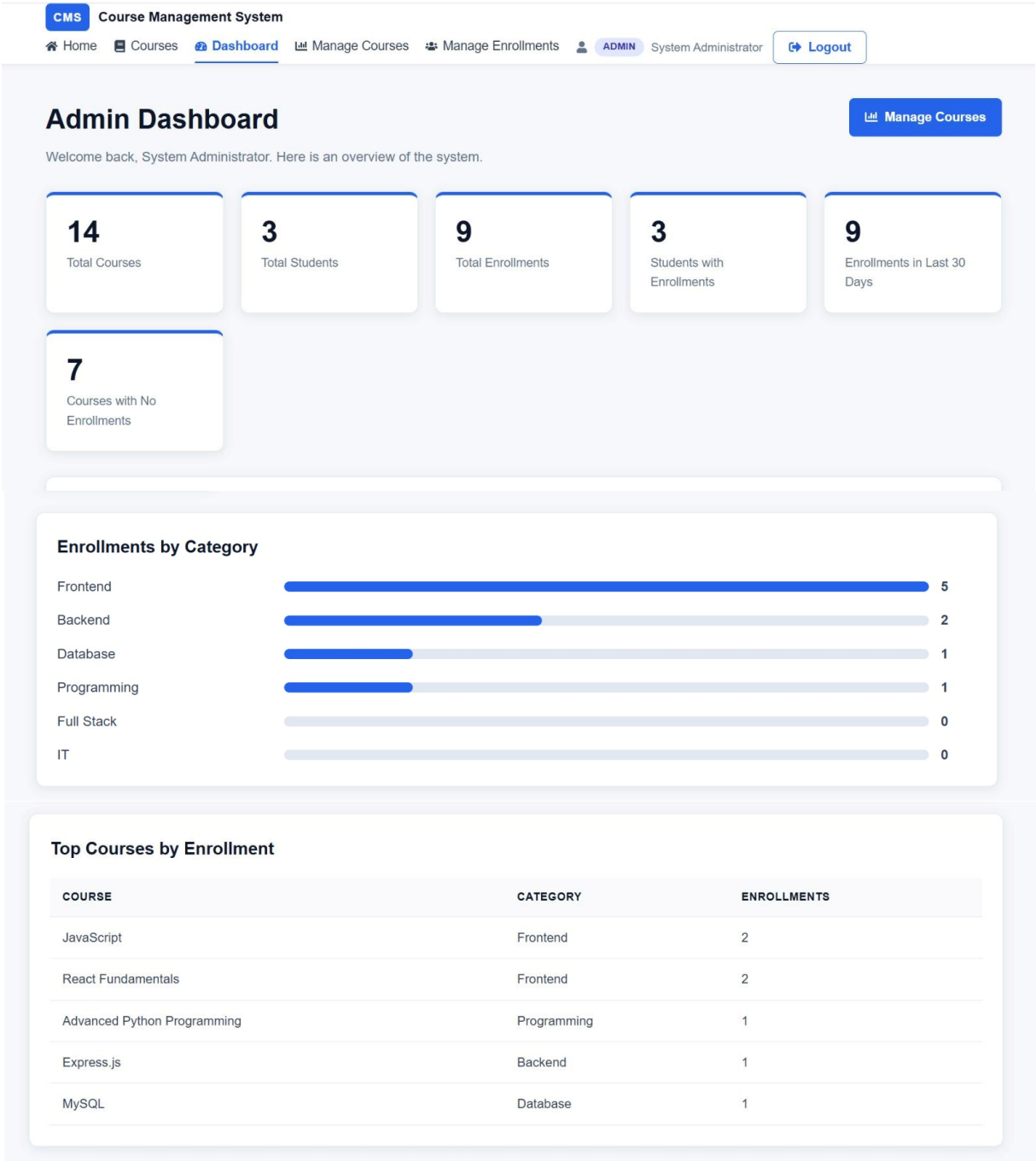
ID	Acceptance Criterion	Verification	Status
AC-001	Admin report provides the required system totals	SQL totals are implemented; comparison against manual SQL was not run	Pass
AC-002	Admin reports are accessible only to administrators	Admin role middleware is applied to the overview route	Pass
AC-003	Student reports are accessible only to students	Student role middleware is applied to the student route	Pass
AC-004	A student can see only their own report	Student queries use the authenticated user ID as a filter	Pass
AC-005	Top courses are sorted by enrollment count	SQL orders by enrollment count descending, with title and ID tie-breakers	Pass
AC-006	Enrollment totals by category are provided	SQL groups enrollment counts by category	Pass
AC-007	Courses without enrollments are counted	SQL counts courses with no matching enrollment rows	Pass
AC-008	Empty report data does not cause errors or show NaN	Safe defaults and empty-state rendering are implemented	Pass
AC-009	Reporting failures do not break the entire dashboard	Dashboards catch report errors and remain rendered	Pass
AC-010	No unnecessary query per course is used	Report model uses a fixed set of aggregate queries	Pass

4. Evidence

Evidence 01 – Reporting Endpoint and Access Control

Description: Source code showing the report endpoints, authentication middleware, and role checks.

TC-
Related: AC-002, AC-003, AC-004



Recent Enrollments

Manage enrollments

STUDENT	USERNAME	COURSE	CATEGORY	LEVEL	ENROLLED ON
Nimal Silva	nimal	React Fundamentals	Frontend	BEGINNER	10/6/2026
Kamal Perera	kamal	React Fundamentals	Frontend	BEGINNER	10/6/2026
Kamal Perera	kamal	Advanced Python Programming	Programming	ADVANCED	10/6/2026
Saman Fernando	saman	React	Frontend	ADVANCED	9/22/2026
Saman Fernando	saman	JavaScript	Frontend	INTERMEDIATE	9/22/2026
Nimal Silva	nimal	Express.js	Backend	ADVANCED	9/22/2026

Final Result

Item	Result
Requirements	Verified
Implementation	Implemented
Testing	Verified
Acceptance Criteria	Verified
Existing CRUD	Verified
Final Status	Verified

4.1 Final Summary

CR-009 was implemented by adding role-protected reporting endpoints for administrators and students. The administrator endpoint returns system-wide course and enrollment statistics, category totals, the top five courses, courses without enrollments, and recent enrollment information. The student endpoint returns a summary of the authenticated student's own learning activity.

The report model uses SQL aggregation and a fixed number of queries. The student report queries are filtered by the authenticated user ID. Both dashboards were updated to use their reporting endpoints, show loading and error messages, and handle empty report values safely.

The changes were committed on branch JFICT15/CR009 as commit 7eb21d3. Automated and live database tests were not run, so runtime verification remains outstanding.

4.2 Issues / Limitations

- The database does not contain an enrollment status field. All enrollment rows are included in the report totals.
- Automated tests, live endpoint tests, and manual SQL comparisons were not run.
- Dashboard screenshots and runtime evidence must be added after running and verifying the application.
- No database schema migration was required.

Student Name: M M Amal

Student ID: JF/ICT/24/15

Date: 22 October 2026

Signature: Amal